



Security Review For Fluent



Collaborative Audit Prepared For:
Lead Security Expert(s):

Fluent
JakubHeba
Koolex

Date Audited:

March 23 - April 9, 2026

Introduction

The Fluent Network is an Ethereum-based Layer 2 blockchain designed to support blended execution across multiple virtual machines within a single execution environment.

Fluent enables developers to build and deploy on-chain applications using the Ethereum Virtual Machine (EVM), Solana Virtual Machine (SVM), and WebAssembly (Wasm), allowing applications written for different virtual machines and programming languages to interoperate atomically on one chain.

This architecture is powered by rWasm, Fluent's unified execution layer, which maintains compatibility with existing developer tooling while reducing fragmentation across blockchain ecosystems. By minimizing execution tradeoffs, Fluent allows developers to use the most appropriate language and virtual machine for each workload without sacrificing composability or security.

Fluent is also designed as a reputation-native execution layer, supporting applications that depend on long-term user behavior and contribution quality. This is enabled through Prints, Fluent's reputation and social capital data layer, and Fluent Connect, a native application that demonstrates how reputation can be used for identity, onboarding, and access control across on-chain applications.

Scope

Repository: [fluentlabs-xyz/solidity-contracts](https://github.com/fluentlabs-xyz/solidity-contracts)

Audited Commit: [08d7562e6107480e3a116cfa625191a2e181b2de](https://github.com/fluentlabs-xyz/solidity-contracts/commit/08d7562e6107480e3a116cfa625191a2e181b2de)

Final Commit: [9421d48e989fb6b866482bfc85fb74124251cfeb](https://github.com/fluentlabs-xyz/solidity-contracts/commit/9421d48e989fb6b866482bfc85fb74124251cfeb)

Files:

- [contracts/bridge/FluentBridge.sol](#)
- [contracts/bridge/FluentBridgeStorageLayout.sol](#)
- [contracts/bridge/L1/L1FluentBridge.sol](#)
- [contracts/bridge/L2/L2FluentBridge.sol](#)
- [contracts/factories/ERC20TokenFactory.sol](#)
- [contracts/factories/GenericTokenFactory.sol](#)
- [contracts/factories/UniversalTokenFactory.sol](#)
- [contracts/gateways/ERC20Gateway.sol](#)
- [contracts/gateways/GatewayBase.sol](#)
- [contracts/gateways/NativeGateway.sol](#)
- [contracts/interfaces/bridge/IFluentBridge.sol](#)

- contracts/interfaces/bridge/IL1FluentBridge.sol
- contracts/interfaces/bridge/IL2FluentBridge.sol
- contracts/interfaces/gateways/IERC20Gateway.sol
- contracts/interfaces/gateways/IGatewayBase.sol
- contracts/interfaces/gateways/INativeGateway.sol
- contracts/interfaces/IGenericTokenFactory.sol
- contracts/interfaces/INitroVerifier.sol
- contracts/interfaces/IRollup.sol
- contracts/interfaces/IRollupTypes.sol
- contracts/interfaces/ISPIVerifier.sol
- contracts/interfaces/IUniversalToken.sol
- contracts/interfaces/oracles/IL1BlockOracle.sol
- contracts/interfaces/oracles/IL1GasOracle.sol
- contracts/libraries/ExcessivelySafeCall.sol
- contracts/libraries/Heap.sol
- contracts/libraries/MerkleTree.sol
- contracts/libraries/Queue.sol
- contracts/oracles/L1BlockOracle.sol
- contracts/oracles/L1GasOracle.sol
- contracts/rollup/Rollup.sol
- contracts/rollup/RollupStorageLayout.sol
- contracts/tokens/ERC20PeggedToken.sol
- contracts/verifier/NitroVerifier.sol

Final Commit Hash

9421d48e989fb6b866482bfc85fb74124251cfeb

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.

- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
3	7	11

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: After `forceRevertBatch` deposits are permanently lost due to the missing re-enqueuing mechanism [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/49>

Vulnerability Detail

When the sequencer calls `acceptNextBatch`, deposit messages are permanently removed from the L1 bridge's FIFO queue. For each block header with a non-zero `depositRoot`, the Rollup contract calls `_checkDeposits`, which pops deposits from the bridge queue via `popSentMessage`:

The `popSentMessage` function calls `Queue.dequeue`, which irreversibly deletes the front element and advances the queue pointer:

If the batch containing these deposits later needs to be reverted via `forceRevertBatch` called by `EMERGENCY_ROLE` when the rollup enters a corrupted state, the `_cleanupForceRevertBatch` function deletes all batch-level storage but makes no call to the bridge to restore the consumed deposits:

After the revert, the deposits are in a limbo state - removed from the bridge queue, so they cannot be included in a future batch, and not part of any finalized batch, since it was reverted. The L1 ETH or tokens remain locked in the bridge contract with no corresponding L2 credit and no refund path. The `L1FluentBridge` contract has no `pushSentMessage`, `re-enqueue`, or `refund` function. The `rollbackMessageWithProof` path requires a finalized batch with a Merkle proof, which is impossible for a reverted batch.

The Rollup contract's `NatSpec` states "No funds are at risk" when describing `forceRevertBatch`, but this claim is incorrect when deposits have been consumed from the queue.

Impact

Direct, permanent loss of user funds. All ETH and tokens deposited by users via the L1 bridge that were consumed during `acceptNextBatch` become permanently locked if the batch is subsequently reverted.

Tool Used

Manual Review

Recommendation

Implement a deposit restoration mechanism that re-enqueues consumed deposits when a batch is reverted. For example, add a `pushSentMessage` function to `L1FluentBridge`,

callable only by the Rollup contract, and call it from `_cleanupForceRevertBatch` for each deposit that was consumed in the reverted batch. This requires storing per-batch deposit data in the `BatchRecord` so the revert logic knows which deposits to restore.

Issue H-2: ERC20 tokens might become permanently locked after failed L2 delivery due to no rollback mechanism [ACKNOWLED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/55>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

When users bridge ERC20 tokens via `ERC20Gateway.sendTokens()`, tokens are escrowed in the gateway contract through `safeTransferFrom`. A cross-chain message is then sent via `L1FluentBridge.sendMessage()` to mint pegged tokens on L2. If L2 delivery fails due to the L2 downtime, contract revert, gateway misconfiguration, or bridge pause, the rollback mechanism only refunds ETH - ERC20 tokens are permanently locked.

The `_rollbackMessage` function in `L1FluentBridge` explicitly drops the message calldata and only sends back the ETH value field:

```
function _rollbackMessage(
    uint256 gasLimit,
    address from,
    address /*to*/,
    uint256 value,
    uint256 /*blockNumber*/,
    uint256 /*messageNonce*/,
    bytes calldata /*message*/,
    bytes32 messageHash
) internal {
    // Guard against the bridge refunding itself - `from` is set to `msg.sender` in
    // `sendMessage`, so this can only happen if the bridge itself originated the
    // → message
    require(from != address(this), ForbiddenSelfCall());

    // `message` is intentionally not forwarded to `from`.
    // The calldata was encoded for `to` on L2 - `from` is the original sender and
    // → was never
    // designed to receive it. Forwarding arbitrary calldata to an unprepared
    // → contract would
    // create an uncontrolled external call and a reentrancy vector.
    // Rollback is ETH-only; ERC-20 or other asset recovery is the responsibility
    // → of a
    // protocol-layer wrapper that exposes its own claimRollback(messageHash)
    // → function.
    (bool success, bytes memory data) =
    // → ExcessivelySafeCall.excessivelySafeCall(from, value, "", gasLimit);
```

For ERC20 bridge transactions, the `value` field is 0 as only the bridge fee uses ETH. The `message calldata` which encodes the `receivePeggedTokens` call with token type and amount is intentionally dropped. The code comment at line 256-257 explicitly acknowledges that a `claimRollback(messageHash)` function should exist on the gateway, but no such function exists on `ERC20Gateway`.

This creates an asymmetry - `NativeGateway` includes a `rescueNative()` function for stuck ETH, but `ERC20Gateway` has no equivalent function. The only path to release escrowed tokens is `receiveOriginTokens()`, which requires `onlyFluentBridge` authorization and a valid L2-to-L1 message what is impossible when the L2 delivery never succeeded.

Impact

Permanent loss of user ERC20 tokens. Any tokens bridged through `ERC20Gateway` that fail delivery on L2 become permanently locked in the gateway contract. The value at risk scales linearly with the total ERC20 volume flowing through the bridge during any L2 failure event. While the gateway is UUPS-upgradeable, allowing the owner to eventually deploy an implementation with a rescue function, users bear the full risk during the governance window with no guaranteed timeline for recovery.

Tool Used

Manual Review

Recommendation

Implement an ERC20 recovery mechanism on `ERC20Gateway`. Two complementary approaches:

1. Add `claimRollback(bytes32 messageHash)` to `ERC20Gateway`. This is the function the code comment already references. It should verify that the corresponding bridge message has been rolled back, decode the original token type and amount from stored message data, and release the escrowed tokens to the original sender.
2. Add `rescueERC20(address token, address to, uint256 amount)` as an owner-only emergency function. Parallel to `NativeGateway.rescueNative()`, this provides a last-resort admin recovery path for tokens stuck due to edge cases not covered by the standard rollback flow.

Issue H-3: Last-moment challenges enable participants to profit from system disruption while recovering deposits and earning fees [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/62>

Summary

Challenge deadlines are calculated from batch acceptance time rather than challenge submission, enabling attackers to submit disputes with insufficient resolution window. The vulnerability allows profitable DoS attacks where challengers incur no deposit loss while claiming rewards and forcing manual emergency recovery. Although documented as intentional design in contract comments, the implementation contains a flaw: challenger deposits ARE refunded on force-revert (contrary to documentation), creating an exploitable economic attack vector.

Vulnerability Detail

The `challengeBlock()` function calculates the challenge deadline as a fixed value from batch acceptance (`acceptedAtBlock + challengeWindow`), regardless of when the challenger submits their dispute. Challengers can submit challenges up until the very last block before the deadline expires, leaving provers with only one block to generate dual proofs (Nitro attestation and SPI ZK proof).

Prover infrastructure latency is inherent and unavoidable: Even with perfect off-chain monitoring of `BlockChallenged` events, generating both Nitro attestation AND SPI ZK proof takes minutes to hours in practice (depending on multiple factors). A challenge submitted at the very end of the window cannot be resolved regardless of prover infrastructure quality or operational practices.

When a challenge cannot be resolved in time:

- The expired challenge remains in the heap with an overdue deadline
- Rollup enters corrupted state when this batch becomes oldest non-finalized via `_rollupCorrupted()`
- `EMERGENCY_ROLE` must call `forceRevertBatch()` to recover
- Challenger's deposit is refunded via `_processForceRevertChallenged()`, but locked until manual intervention

Impact

A malicious challenger can trigger rollup corruption without losing their deposit (which gets refunded on force-revert) while still claiming the reward fee from successful

challenge resolution. This creates an economic attack vector where attackers profit from causing systemic disruption that requires manual emergency recovery for every incident, despite contract comments stating deposits remain locked and unrecovered.

Code Snippet

```
// challengeBlock()
uint256 deadline = uint256(batch.acceptedAtBlock) + $_challengeWindow; // Fixed
↳ from acceptance!
require(block.number < uint256(batch.acceptedAtBlock) + $_challengeWindow,
↳ ChallengeTooLate(batchIndex));

// resolveChallenge() - dual verification in _verifyNitroAndSp1
// Verification path 1: Nitro enclave attestation - proves the TEE processed this
↳ block
    // External call to a trusted (whitelisted) verifier; reverts if signature
    ↳ is invalid
    INitroVerifier(nitroVerifier).verifyBlock(
        blockHeader.previousBlockHash,
        blockHeader.blockHash,
        blockHeader.withdrawalRoot,
        blockHeader.depositRoot,
        nitroSignature,
        blobHashes
    );
    // Verification path 2: SP1 ZK proof - mathematically proves block
    ↳ execution correctness
    // Both paths must succeed for a challenge to be resolved
    _proveBlockWithSp1(sp1Verifier(), blobHashes, blockHeader, sp1Proof);

// _rollupCorrupted() - triggers corruption when challenge deadline exceeded
    if (status == BatchStatus.Challenged && !$_challengeQueue.isEmpty()) {
        return $_challenges[$_challengeQueue.peek()].deadline < block.number;
    }

// Contract comments state "This is by design" but:
// - Comment claims deposit "remains locked...and is not returned"
// - Actual code refunds deposit + incentive fee to challengerRewards on
↳ force-revert
// - No on-chain guard prevents last-moment submission (only checks strict
↳ less-than)
// - Even with perfect monitoring, proof generation takes minutes-hours not 12
↳ seconds
```

Tool Used

Manual Review

Recommendation

Calculate challenge deadline dynamically based on submission time (`block.number + resolutionWindow`) to ensure provers always have a fixed window regardless of when challenger submits. Alternatively, enforce minimum time remaining before allowing new challenges (e.g., reject challenges submitted within last X blocks where X provides sufficient time for dual proof generation).

Issue M-1: Retroactive protocol parameter changes affect in-flight operations [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/50>

Vulnerability Detail

All timing and deadline parameters in the Rollup and bridge contracts are stored as global values read at operation-execution time rather than snapshotted when an operation is first accepted. This means an admin parameter change takes effect retroactively on operations that were already accepted, queued, or in transit under the original parameter values.

The `_tryFinalizeBatch` function illustrates the pattern:

The `BatchRecord` struct stores `acceptedAtBlock` but no timing snapshot. Similarly, `_checkDeposits` reads the deadline at call time:

And `_beforeReceiveMessage` on L2 reads the current deadline at message delivery time:

The setter for `finalizationDelay` has only one constraint - the new value must exceed the current `challengeWindow`. There is no lower bound relative to in-flight batches, and there is no grace period:

Impact

When `setFinalizationDelay` is called while batches are in `Preconfirmed` state, those batches immediately become eligible for finalization if they have aged past the new (shorter) delay, even though they were accepted under a much longer delay.

When `setAcceptDepositDeadline` is called while deposits are in the FIFO queue, any deposit that was within the original deadline but exceeds the new (shorter) deadline instantly becomes "expired." The next call to `acceptNextBatch` that encounters such a deposit will revert with `AcceptDepositDeadlineExceeded`, and because the queue is FIFO with no skip mechanism, all subsequent deposits behind the expired entry are also blocked until the deadline is extended.

When `setReceiveMessageDeadline` is called on `L2FluentBridge` while messages are in transit from L1, any message whose age exceeds the new (shorter) deadline is instantly marked `Failed`. The `receiveFailedMessage` retry path also calls `_beforeReceiveMessage`, which applies the same reduced deadline, so retry also fails.

Tool Used

Manual Review

Recommendation

Snapshot timing parameters at the moment of operation acceptance and use the snapshotted value for all subsequent checks on that operation.

1. Add `finalizationDelay`, `acceptDepositDeadline`, and any other timing parameter to the relevant per-operation record and populate them at acceptance time.
2. In `_tryFinalizeBatch`, compare against `batch.finalizationDelaySnapshot` rather than `$_finalizationDelay`.
3. In `_checkDeposits`, compare against the delay recorded at deposit enqueue time rather than the current global value.
4. For `receiveMessageDeadline` on L2, include the deadline in the message commitment so L2 delivery uses the deadline that was in effect on L1 at send time.

Issue M-2: The `incentiveFee` overflow and `PROVER` role revocation may lead to temporary system freeze [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/51>

Vulnerability Detail

Two individually legitimate admin operations, so setting the `incentiveFee` parameter and revoking `PROVER_ROLE` can combine to simultaneously block all paths for clearing a corrupted rollup state.

The `_setIncentiveFee` setter accepts any `uint256` with no upper bound:

When `forceRevertBatch` is called, the contract accumulates challenger rewards at line 175:

If `incentiveFee` is set to `type(uint256).max`, this addition overflows, causing every `forceRevertBatch` call to revert with a `Panic`. At the same time, if `PROVER_ROLE` has been revoked, `resolveChallenge` is gated at:

With both paths blocked simultaneously, the rollup remains in a permanently corrupted state as `isRollupCorrupted() == true` and cannot process new batches or finalize.

The combination can occur during routine operations as the admin may update `incentiveFee` as part of economic parameter tuning, and `PROVER_ROLE` may be revoked during a key rotation. If both happen while challenges are active, the freeze materialises.

Impact

The rollup cannot process new batches, finalize existing ones, or clear corruption while both conditions hold. If the admin role is controlled by a timelock governance contract, the freeze lasts for the full timelock duration.

Tool Used

Manual Review

Recommendation

Add an upper bound check to `_setIncentiveFee`: for example, `require(newIncentiveFee <= MAX_INCENTIVE_FEE)` where `MAX_INCENTIVE_FEE` is a reasonable cap. Consider separating the incentive fee accumulation from the overflow-sensitive arithmetic by capping the credited amount per challenge rather than accumulating unbounded values.

Issue M-3: Zero depositRoot batches bypass deposit processing [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/52>

Vulnerability Detail

The `acceptNextBatch` only invokes `_checkDeposits` when a block header explicitly claims deposits:

The sequencer controls block header construction. By setting `depositRoot == ZERO_BYTE S_HASH` and `depositCount == 0` in every submitted block header, the sequencer can advance the rollup state indefinitely by submitting, preconfirming, and finalizing batches, without ever processing pending deposits. The bridge queue accumulates unprocessed deposits until they exceed `_acceptDepositDeadline`.

When the stale deposit at the head of the FIFO queue has aged past the deadline, any future batch that does include deposits will revert at `_checkDeposits`:

Because `Queue.sol` has no `skip`, `remove`, or `clear` function, the expired head deposit permanently blocks all successors until admin governance action extends the deadline.

So, `_rollupCorrupted()` does not monitor deposit queue liveness. It checks blob submission windows, preconfirmation deadlines, and challenge deadlines, but the rollup reports as healthy during an active censorship campaign:

This means automated monitoring based on `_rollupCorrupted()` will not alert on deposit censorship, and the `forceRevertBatch` recovery path, which requires `_rollupCorrupted()` to return true, is unavailable.

Impact

A compromised or malicious `SEQUENCER_ROLE` can prevent all L1-to-L2 deposits from being processed without any on-chain signal. Users observe that their deposits never arrive on L2, but the rollup itself reports no corruption. Once the queue head expires, all future deposit-bearing batches fail, blocking new and pending depositors alike until a governance action extends the deadline.

Code Snippet

Tool Used

Manual Review

Recommendation

Add deposit liveness to `_rollupCorrupted()`, so if the bridge queue is non-empty and the head deposit's age exceeds `_acceptDepositDeadline`, treat the rollup as degraded. This enables `forceRevertBatch` as a recovery path and triggers automated alerts.

Additionally, consider requiring that each batch either includes all pending deposits whose deadline is within a configurable window, or explicitly provides a signed attestation that no deposits are pending. This makes censorship expensive and detectable.

Issue M-4: NitroVerifier signed payload lacks chain and contract binding [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/56>

Vulnerability Detail

The NitroVerifier computes TEE-attested signatures over payloads that contain no deployment-specific binding. There is no `block.chainid`, no `address(this)`, and no other identifier that ties the signature to a particular chain or contract instance.

The `verifyBatch` computes:

```
function verifyBatch(bytes32 batchRoot, bytes32[] calldata blobHashes, bytes
↳ calldata signature) external view returns (address) {
    require(signature.length == 65, InvalidSignatureLength());

    bytes32 payload = sha256(abi.encode(batchRoot, blobHashes));
    address verifier = _assertSignerAttested(payload, signature);

    return verifier;
}
```

Neither hash includes any deployment identity. The `INitroVerifier` interface comment acknowledges only same-chain replay protection "Replay protection (blockHash / batchRoot deduplication) is the caller's responsibility" but explicitly makes cross-deployment domain binding a non goal. The caller at `Rollup.sol:351` adds no chain binding either:

```
// External call to the Nitro verifier: validates the enclave attestation signature
// over the batch root and blob hashes; returns the signer address for the event
address verifier = INitroVerifier(nitroVerifier).verifyBatch(batch.batchRoot,
↳ $_batchBlobHashes[batchIndex], signature);
```

A valid TEE enclave signature produced on one Fluent deployment is byte-for-byte identical and cryptographically valid on any other deployment where the same enclave pubkey is attested and the same (batchRoot, blobHashes) tuple appears.

Impact

A TEE enclave signature produced on one deployment can be replayed to preconfirm a batch on a different deployment without the enclave actually processing that deployment's batch data. This undermines the fundamental integrity guarantee of the TEE-based preconfirmation, that only batches specifically validated by the enclave for this chain advance to `Preconfirmed` state. When combined with the empty Merkle proof

behavior for single-block batches, cross-deployment replay could in principle be chained toward unauthorized withdrawal proof acceptance on the target deployment.

Tool Used

Manual Review

Recommendation

Include `block.chainid` and `address(this)` - the NitroVerifier contract address - in the hashed payload for both `verifyBatch` and `verifyBlock`

Issue M-5: The deposit censorship, FIFO Lock and forceRevert non-restoration chain [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/57>

Vulnerability Detail

Three individually confirmed issues compose into a chain that can temporarily disable all L1-to-L2 deposit processing and, in a specific scenario, permanently lose individual user deposits.

1. Sequencer censors deposits via zero-deposit batches

The `acceptNextBatch` only calls `_checkDeposits` when a block header claims deposits:

```
for (uint256 i = 0; i < batchSize; ++i) {
    // Only check blocks that claim deposits - skip blocks with empty deposit roots
    if (blockHeaders[i].depositRoot != ZERO_BYTES_HASH)
        ↪ _checkDeposits(blockHeaders[i]);
}
```

The sequencer (`SEQUENCER_ROLE`) controls block header construction and can submit an unlimited number of batches where every header has `depositRoot == ZERO_BYTES_HASH`. Queued deposits age unconsumed. The rollup's `_rollupCorrupted()` function does not monitor deposit queue liveness so the system reports as healthy throughout the censorship period.

2. Stale deposit at queue head blocks all successors

The `Queue.sol` is a strict FIFO library with no `skip`, `remove`, or `indexed-access` functions. When `_checkDeposits` eventually encounters the aged deposit, the deadline check reverts:

```
for (uint256 i = 0; i < header.depositCount; ++i) {
    // External call to the bridge: pops the next deposit from the FIFO queue
    // Called after all state writes in acceptNextBatch (CEI) under nonReentrant
    ↪ guard
    (bytes32 depositId, uint256 depositBlockNumber) =
    ↪ IL1FluentBridge($.bridge).popSentMessage(); // wake-disable-line reentrancy
    // Ensure the deposit is not stale - prevents sequencers from including very
    ↪ old deposits
    require(block.number <= depositBlockNumber + deadline,
    ↪ AcceptDepositDeadlineExceeded(depositBlockNumber + deadline, block.number));
    depositIds[i] = depositId;
}
```

The revert undoes the pop, leaving the expired deposit at the queue head. All subsequent deposits enqueued behind it are unreachable so every attempt to include a

deposit-bearing batch will revert at the expired front entry.

3. Deposits consumed by a reverted batch are permanently lost

If `forceRevertBatch` (`EMERGENCY_ROLE`) is called on a batch that had already popped deposits via `_checkDeposits`, those deposits are gone. `_cleanupForceRevertBatch` deletes only Rollup-side state:

```
function _cleanupForceRevertBatch(uint256 batchIndex) internal {
```

The `Queue.sol` has no `pushFront` or re-enqueue function. Deposits consumed before the revert are unrecoverable at the contract level.

Because `_checkDeposits` reads `$_acceptDepositDeadline` from current global storage at execution time, an admin can call `setAcceptDepositDeadline(type(uint32).max)` to retroactively un-expire the stale front entry. After this, a sequencer can include the formerly-expired deposit, the queue unblocks, and the deadline can be restored. This prevents permanent bridge queue destruction but requires timely governance action. During the bricking period all L1-to-L2 deposits are unavailable, however deposits already consumed by a reverted batch are not recoverable via this path.

Impact

- Temporary loss of deposit processing capability for all users, persisting until admin extends the deadline.
- Permanent and irrecoverable loss of individual user deposits if `forceRevertBatch` is called on a batch that already popped deposits from the bridge queue.
- No fee or compensation mechanism exists for affected depositors.

Tool Used

Manual Review

Recommendation

1. In `_cleanupForceRevertBatch`, add a call to re-enqueue any deposits that were consumed by the reverted batch. This requires either a `pushFront` function in `Queue.sol` or a bridge-level re-enqueue API.
2. The `_rollupCorrupted` function currently checks blob submission, preconfirmation, and challenge deadlines but not deposit liveness. We recommend adding a check that if the queue is non-empty and the head deposit has exceeded its deadline, the rollup is in a degraded state requiring intervention.
3. Rather than relying on the deadline as a purely reactive punitive mechanism, consider requiring that batches include pending deposits if any have been in the queue for more than a configurable fraction of the deadline

Issue M-6: The `receiveMessageWithProof` skips nonce increment [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/58>

Vulnerability Detail

The L1 bridge supports two parallel paths for delivering L2-to-L1 cross-chain messages - a proof-based path (`receiveMessageWithProof`) and a relayer path (`receiveMessage`). Both paths are intended to coexist, with the proof path as the preferred production mechanism and the relayer path as a transitional fallback.

The `receiveMessage` enforces sequential nonce ordering via `_takeNextReceivedNonce()`:

```
function receiveMessage(
    address from,
    address to,
    uint256 value,
    uint256 chainId,
    uint256 blockNumber,
    uint256 messageNonce,
    bytes calldata message
) external onlyRole(RELAYER_ROLE) nonReentrant whenNotPaused {
    require(messageNonce == _takeNextReceivedNonce(),
        → MessageReceivedOutOfOrder());
```

The `receiveMessageWithProof` delivers the same message via Merkle proof but does not call `_takeNextReceivedNonce()`:

```
function receiveMessageWithProof(
    uint256 batchIndex,
    L2BlockHeader calldata blockHeader,
    address from,
    address payable to,
    uint256 value,
    uint256 chainId,
    uint256 blockNumber,
    uint256 messageNonce,
    bytes calldata message,
    MerkleTree.MerkleProof calldata withdrawalProof,
    MerkleTree.MerkleProof calldata blockProof
) external nonReentrant whenNotPaused {
[.]
    // Execute the cross-chain message with gasleft(), forwarding remaining gas
    // _receiveMessage records the outcome (Success/Failed) in storage
    _receiveMessage(gasleft(), from, to, value, message, messageHash);
}
```

After a single proof-path delivery of message nonce `N`, `_receivedNonce` remains at `N`. The relayer then enters a permanent deadlock:

1. Relayer attempts nonce `N` via `receiveMessage` - the nonce check passes (`N == N`), but `MessageAlreadyReceived` reverts because `messageHash` is already recorded as `Success`. The revert rolls back `_takeNextReceivedNonce()`, leaving `_receivedNonce` still at `N`.
2. Relayer attempts nonce `N+1` via `receiveMessage` - `MessageReceivedOutOfOrder` reverts because `_receivedNonce` is `N`, not `N+1`.

The relayer path is permanently blocked after a single proof-path delivery. No admin function or recovery path exists to synchronize `_receivedNonce` to the actual number of received messages.

Impact

Permanent operational degradation of the relayer delivery path. Messages that require relayer delivery cannot be delivered. No direct fund loss occurs because the proof path itself is unaffected and messages remain provably claimable. However, the documented dual-path architecture is broken after the first mixed-mode operation, and there is no recovery mechanism other than a contract upgrade.

Tool Used

Manual Review

Recommendation

The `receiveMessageWithProof` should call `_takeNextReceivedNonce()` before recording the message, or otherwise advance the `_receivedNonce` counter to reflect proof-path deliveries.

If the proof path and relayer path are intended to be fully interchangeable and concurrent, they must share a consistent view of which nonces have been consumed. If the two paths are intended to be mutually exclusive, this should be enforced in code and documented explicitly in the `SecurityModel`.

Issue M-7: Zero expected blobs creates unreachable submit state [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/63>

Summary

The `acceptNextBatch` function accepts `expectedBlobsCount = 0` because there's no lower bound validation on the input. This lets batches enter `HeadersSubmitted` status with zero expected blobs, but `submitBlobs` can never successfully execute against that requirement. The batch stays stuck indefinitely, blocking all subsequent finalization until emergency intervention forces a revert.

Vulnerability Detail

When someone with `SEQUENCER_ROLE` calls `acceptNextBatch`, they pass in how many blobs the batch expects to submit next. The function only validates that this number doesn't overflow (`<= type(uint32).max`), so zero passes through without complaint. That batch gets marked as `HeadersSubmitted` and stored with `batch.expectedBlobs = 0`. Later, when `submitBlobs` is called to actually deliver those blobs, it has two requirements that can never both be true at the same time:

- The first requires at least one blob (`numBlobs > 0`)
- The second requires the total not exceed what was promised (`blobHashes.length + numBlobs <= batch.expectedBlobs`) With `expectedBlobs = 0`, the second condition forces the total to be zero, but the first explicitly forbids that. The transaction reverts, status never changes from `HeadersSubmitted`, and since `_tryFinalizeBatch` enforces strict sequential ordering through preconfirmation states, every single batch behind this one is permanently blocked.

Impact

A compromised or misconfigured `SEQUENCER_ROLE` can create a permanent blockage with no automated recovery path. All L2-to-L1 withdrawals halt for the entire chain until `EMERGENCY_ROLE` intervenes. If deposits were consumed during acceptance of that stuck batch, those funds are popped from the bridge queue and never returned on revert.

Code Snippet

```
// acceptNextBatch - only upper bound checked (missing lower bound)
require(expectedBlobsCount <= type(uint32).max,
    ↪ ExpectedBlobsCountOverflow(expectedBlobsCount));
batch.expectedBlobs = uint32(expectedBlobsCount); // accepts 0
```

```
// submitBlobs - mutually exclusive when expectedBlobs=0
require(numBlobs > 0, ZeroValueNotAllowed("numBlobs"));           // requires >=1
require(blobHashes.length + numBlobs <= batch.expectedBlobs, ...); // requires <=0
↪ for expectedBlobs=0
```

Tool Used

Manual Review

Recommendation

Add validation to reject zero values for `expectedBlobsCount` at the entry point of `acceptN` `extBatch`. This prevents batches from being created with invalid blob expectations that cannot be fulfilled by subsequent submission.

Issue L-1: Interior block headers with zero root and non-zero depositCount pass validation unchecked [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/53>

Vulnerability Detail

When a sequencer submits batch headers via `acceptNextBatch`, the zero-deposit-root consistency check is applied only to the last block header in the batch (`blockHeaders[batchSize - 1]`):

```
// Sanity check: a zero deposit root must have zero deposit count (no phantom
↪ deposits)
if (blockHeaders[batchSize - 1].depositRoot == ZERO_BYTES_HASH) {
    require(blockHeaders[batchSize - 1].depositCount == 0,
    ↪ InvalidDepositRootWithNonZeroCount(blockHeaders[batchSize -
    ↪ 1].depositCount));
}
```

Interior headers are not subject to this check. A sequencer could submit a batch where interior blocks have `depositRoot == ZERO_BYTES_HASH` but `depositCount > 0`, representing a logically inconsistent state. The `_checkDeposits` loop at L275-278 only processes blocks where `depositRoot != ZERO_BYTES_HASH`, so these phantom deposit counts are silently ignored rather than rejected.

Impact

No direct fund loss as inconsistent interior headers do not cause deposit double-counting or fund theft. However, off-chain monitors and indexers parsing `depositCount` fields to track bridge usage could be misled by phantom deposit counts in interior blocks.

Tool Used

Manual Review

Recommendation

Extend the zero-root check to all block headers in the batch.

Issue L-2: The `setL1BlockNumber` accepts 0 and `MAX_UINT` [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/54>

Vulnerability Detail

The `L1BlockOracle.setL1BlockNumber()` function called by the L2 bridge accepts any `uint256` value including 0 and `type(uint256).max` with no bounds validation:

- If `value = 0`, all L2 message deadline checks that compare against `l1BlockNumber` would treat every message as if the L1 block is at genesis. Messages with any `deadline > 0` would appear permanently in-flight (never expired), but messages using `l1BlockNumber` to verify delivery timing would behave incorrectly.
- if `value = type(uint256).max`, all deadline comparisons using `block.number <= deadline` would instantly make every message appear as permanently expired. New messages cannot be delivered, so existing in-flight messages cannot be processed.

Impact

An admin error or compromise on L2 could halt all L2 message delivery. Recovery requires a corrective `setL1BlockNumber` call.

Tool Used

Manual Review

Recommendation

Add bounds validation to `setL1BlockNumber`, rejecting 0 and values implausibly far from the current known L1 block.

Issue L-3: Admin parameter setters lack bounds validation [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/59>

Vulnerability Detail

Five admin setters in `RollupStorageLayout` accept values that, while passing their minimal validation checks, place the protocol in a degraded or dysfunctional state. Each setter is guarded by `DEFAULT_ADMIN_ROLE`, so exploitation requires a privileged actor.

1. The `_setIncentiveFee` accepts any `uint256` with no upper bound. At `forceRevertBatch`, the loop computes `challenge.deposit + fee` for each challenged block. When `fee = type(uint256).max`, this addition overflows, causing `forceRevertBatch` to revert. All batch recovery is blocked until the admin calls `setIncentiveFee(0)`.

```
function _setIncentiveFee(uint256 newIncentiveFee) internal {
    RollupStorage storage $ = _getRollupStorage();
    emit IncentiveFeeUpdated($._incentiveFee, newIncentiveFee);
    $_incentiveFee = newIncentiveFee;
}
```

2. The `_setChallengeWindow` only checks `newChallengeWindow < _finalizationDelay`. A value of 0 passes when `_finalizationDelay > 0` because `0 < delay`. Zero challenge window means the expression `block.number < batch.acceptedAtBlock + 0` is always false so challengers can never submit a dispute.
3. The `_setChallengeDepositAmount` only checks `> 0`. Setting to 1 wei eliminates the economic barrier against spam challenges.
4. The `_setGasLeft` only checks `!= 0`. Values above ~30,000,000 (Ethereum block gas limit) make `gasleft() > _gasLeft` never true inside `acceptNextBatch`, permanently preventing batch acceptance.
5. The `_setAcceptDepositDeadline` only checks `!= 0`. Setting to `type(uint32).max` effectively disables the deposit deadline.

Impact

A `DEFAULT_ADMIN_ROLE` key compromise or misconfiguration can silently disable the challenge mechanism, block all batch recovery, enable low-cost challenge spam, brick batch acceptance, or disable deposit deadlines.

Code Snippet

Tool Used

Manual Review

Recommendation

Add meaningful bounds to each setter, for example:

- `incentiveFee` - maximum 100 ETH
- `challengeWindow` - minimum 1 block
- `challengeDepositAmount` - minimum 0.001 ETH
- `gasLeft` - maximum 30,000,000
- `acceptDepositDeadline` - maximum 50,400 blocks

Issue L-4: The `_validateChallenge` does not verify supplied `batchIndex` matches challenged batch [RE-SOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/60>

Vulnerability Detail

The `resolveChallenge` accepts a caller-supplied `batchIndex` alongside the block header to be proven. The internal `_validateChallenge(batchIndex, commitment, blockProof)` call verifies that `commitment` exists in the Merkle tree rooted at `batchRoot` for the supplied `batchIndex`, but it does not verify that this `batchIndex` matches the `batchIndex` stored in the challenge record for `commitment`:

```
function resolveChallenge(
  uint256 batchIndex,
  L2BlockHeader calldata blockHeader,
  MerkleTree.MerkleProof calldata blockProof,
  address nitroVerifier,
  bytes calldata nitroSignature,
  bytes calldata sp1Proof
) external nonReentrant whenNotPaused onlyRole(PROVER_ROLE) {
  // Access control: PROVER_ROLE - only authorized provers can resolve
  → challenges
  RollupStorage storage $ = _getRollupStorage();
  // Cannot resolve challenges when the rollup is corrupted (must
  → force-revert first)
  require(!_rollupCorrupted(), RollupCorrupted());

  // Derive the commitment from the block header - this uniquely identifies
  → the block
  bytes32 commitment = _computeCommitment(blockHeader);

  // Validate: challenge exists, not yet proven, and block is in the batch
  → Merkle tree
  _validateChallenge(batchIndex, commitment, blockProof);
  // Dual verification: both Nitro enclave attestation AND SP1 ZK proof must
  → pass
  _verifyNitroAndSp1(batchIndex, blockHeader, nitroVerifier, nitroSignature,
  → sp1Proof);

  ChallengeRecord storage challenged = $_challenges[commitment];
  // Cache the deposit amount before deleting the challenge record
  uint256 deposit = challenged.deposit;

  // Effects before interactions (CEI): credit reward before deleting
  → challenge state
```

```

// Mark the block as proven so it cannot be challenged again
$. _provenBlocks[commitment] = true;
// Track proven blocks per batch - used to detect when all challenges are
→ resolved
$. _batchProvenBlocks[batchIndex].push(commitment);
// Transfer the challenger's deposit to the prover as a reward
// (credited, not transferred - prover withdraws via withdrawProofReward)
$. _proverRewards[_msgSender()] += deposit;

// Clean up: delete the challenge record and remove from the priority queue
delete $. _challenges[commitment];
_removeChallengeFromQueue(commitment);

// When every challenged block in the batch has been proven, restore to
→ Preconfirmed
// so the batch can proceed toward finalization
if ($. _batchChallengedBlocks[batchIndex].length ==
→ $. _batchProvenBlocks[batchIndex].length) {
    $. _batches[batchIndex].status = BatchStatus.Preconfirmed;
}
// Log resolution - off-chain systems track this to update batch confidence
→ scores
emit ChallengeResolved(batchIndex, commitment, _msgSender());
}

```

The ChallengeRecord at \$. _challenges[commitment] contains the correct batchIndex that the challenge was opened against. If a prover supplies a different batchIndex while the challenge is recorded against a different batch, and if the block happens to exist in both batches Merkle trees, then \$. _batchProvenBlocks for the wrong batch could receive an entry.

In practice, Merkle tree membership proofs prevent cross-batch confusion, as a block commitment from batch Y will not satisfy the Merkle inclusion proof against batch X's root unless the batch roots happen to be identical. No fund loss is possible under the current trust model.

Impact

In edge cases, the proven-block counter for the wrong batch could be incremented. The implicit Merkle proof protection makes this exploitable only with very specific batch root collisions.

Tool Used

Manual Review

Recommendation

Add an explicit check in `_validateChallenge` or at the start of `resolveChallenge`:

```
ChallengeRecord storage record = $_challenges[commitment];  
require(record.batchIndex == batchIndex, InvalidBatchIndex(batchIndex,  
↪ record.batchIndex));
```

Issue L-5: The OwnableUpgradeable single-step transfer in use [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/61>

Vulnerability Detail

The ERC20PeggedToken inherits OwnableUpgradeable single-step ownership transfer:

```
contract ERC20PeggedToken is Initializable, ERC20Upgradeable, OwnableUpgradeable,  
    ↪ PausableUpgradeable, ERC165Upgradeable {  
    /// @notice Token transfer attempted while paused.
```

The OwnableUpgradeable.transferOwnership(newOwner) immediately sets the new owner without requiring the recipient to accept. A single typo in transferOwnership(wrongAddress) permanently and irrecoverably transfers ownership of the pegged token to an address that may not exist, be a smart contract that cannot call transferOwnership back, or be an attacker.

Since the gateway (owner) controls mint(), burn(), and pause() on all pegged tokens, irrecoverable ownership loss means the gateway can no longer mint or burn tokens for that pegged token, bridging of the affected asset is permanently broken.

The GenericTokenFactory itself correctly uses Ownable2StepUpgradeable, highlighting the inconsistency.

Impact

A single mistaken transferOwnership call permanently breaks bridging for the affected pegged token. Existing holders cannot withdraw their tokens back to L1 via the normal bridge path. Recovery requires a new pegged token deployment and a migration process for existing holders.

Tool Used

Manual Review

Recommendation

Inherit Ownable2StepUpgradeable instead of OwnableUpgradeable.

Issue L-6: Duplicate message hash computation in L2 receive path [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/64>

Vulnerability Detail

The same `messageHash` is computed twice on every L2 message receipt - once in the base `FluentBridge.receiveMessage` and once in the `L2FluentBridge._beforeReceiveMessage` override. Both call sites invoke `keccak256(_encodeMessage(...))` with identical arguments.

`FluentBridge.sol:L86:`

```
bytes32 messageHash = keccak256(_encodeMessage(from, to, value, chainId,  
↪ blockNumber, messageNonce, message));
```

`L2FluentBridge.sol:L152:`

```
bytes32 messageHash = keccak256(_encodeMessage(from, to, value, chainId,  
↪ blockNumber, messageNonce, message));
```

The `_encodeMessage` performs a full `abi.encode` of 7 parameters, including a dynamic bytes, and the `keccak256` runs over the encoded blob.

The same pattern applies to `receiveFailedMessage`, which also computes `messageHash` before calling `_beforeReceiveMessage`.

Impact

Every inbound L2 message pays the duplicate hashing cost. This path covers all L1 -> L2 deposits and cross-chain messages delivered to L2 users via the relay. The cost compounds linearly with message throughput, a rollup processing thousands of messages per day leaves meaningful gas on the table for relayers and indirectly for users through the fee model.

Tool Used

Manual Review

Recommendation

Change the hook signature to accept the already-computed `messageHash`.

Issue L-7: Double load of GasPriceConfig struct in fee calculation [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/65>

Vulnerability Detail

The L2 send-fee path loads the full GasPriceConfig struct (3 storage slots) from storage twice on every call. getSendMessageFee calls getL1GasLimit() * _calculateGasCost(), and both of those internal calls independently invoke getGasPriceConfig():

```
/// @inheritdoc IFluentBridgeRead
function getSendMessageFee() public view override returns (uint256) {
    // Total fee = assumed L1 gas units * per-unit cost (scalar-adjusted L1 price +
    //   ↪ overhead)
    return getL1GasLimit() * _calculateGasCost();
}

function getL1GasLimit() public view returns (uint256) {
    // Reads via getGasPriceConfig() which loads the full struct into memory
    return getGasPriceConfig()._l1GasLimit;
}

/** @dev Computes the L1 gas cost component of the send fee using oracle price and
    ↪ config. */
function _calculateGasCost() internal view returns (uint256) {
    // Cache config in memory to avoid repeated SLOAD
    GasPriceConfig memory gasPriceConfig = getGasPriceConfig();
    // Query the oracle for the current L1 base fee (in wei per gas unit)
    uint256 l1GasPrice = IL1GasOracle(getL1GasPriceOracle()).getL1GasPrice();
    // Formula: (l1GasPrice * scalarGasPrice) / 1e18 + overheadGasPrice
    // 1e18 denominator enables fractional scaling (e.g. 1.5x = 1.5e18)
    // overheadGasPrice adds a fixed base cost independent of L1 conditions
    return ((l1GasPrice * gasPriceConfig._scalarGasPrice)) / 1e18 +
        ↪ gasPriceConfig._overheadGasPrice;
}
```

The compiler cannot CSE across public view function boundaries, so the optimized IR shows getGasPriceConfig() being lowered twice, each load copying all 3 uint256 slots into memory. That is 6 SLOADs plus two memory allocations where 3 SLOADs and one allocation suffice.

Impact

The getSendMessageFee runs on every L2 sendMessage call, which is the primary path for L2 -> L1 value transfer and gateway-based withdrawals. Every user withdrawing assets from

L2, native ETH through `NativeGateway` and ERC20 tokens through `ERC20Gateway`, pays this overhead directly as part of the required fee check.

Tool Used

Manual Review

Recommendation

Inline the fee calculation so the config is loaded once. Using `storage` instead of `memory` for `config` avoids the struct copy and lets the compiler issue field-level SLOADs only when needed.

Issue L-8: Uncached external contract addresses in ERC20Gateway.sendTokens [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/66>

Vulnerability Detail

The `sendTokens` reads `getOtherSideGateway()` twice (L101 and L128) and `getBridgeContract()` twice (L109 and L128). Between those reads, the function executes `_sendOriginTokens` or `_sendPeggedTokens`, both of which perform external calls to arbitrary ERC20 token contracts (`safeTransferFrom`, `burn`).

Because the token contract is untrusted, the compiler cannot assume the gateway's own storage remains unchanged across those calls, and `via-ir` keeps the second set of SLOADs in the optimized IR.

```
function sendTokens(address token, address to, uint256 amount) external payable
↳ nonReentrant {
    // Ensure the remote gateway has been configured - cannot route messages
    ↳ without a destination
    require(getOtherSideGateway() != address(0),
    ↳ ZeroAddressNotAllowed("getOtherSideGateway"));
    // Prevent accidental burns to the zero address on the destination chain
    require(to != address(0), InvalidRecipient());
    // Prevent sending zero tokens - disallow no-op messages that still consume
    ↳ bridge fees
    require(amount > 0, ZeroValueNotAllowed("amount"));
    // Exact fee required - excess ETH would become cross-chain native value, but
    ↳ receive
    // functions (receivePeggedTokens/receiveOriginTokens) are not payable, so
    ↳ delivery
    // would permanently fail and lock both the ETH and the bridged tokens
    require(msg.value == FluentBridge(getBridgeContract()).getSentMessageFee(),
    ↳ ExactFeeRequired());

    // Cache msg.sender to pass as both the protocol-level sender and the token
    ↳ source
    address sender = msg.sender;
    bytes memory message;

    // Determine token mode by checking whether a pegged-to-origin mapping exists.
    // No mapping means this is an origin token on the current chain (deposit path:
    ↳ escrow locally).
    // A mapping means this is a pegged token on the current chain (withdraw path:
    ↳ burn locally).
    if (getTokenMapping(token) == address(0)) {
        // Origin token path: lock tokens in this gateway and encode a
        ↳ receivePeggedTokens call
```

```

        message = _sendOriginTokens(token, sender, sender, to, amount);
    } else {
        // Pegged token path: burn the pegged representation and encode a
        ↪ receiveOriginTokens call
        message = _sendPeggedTokens(token, sender, sender, to, amount);
    }

    // Forward the encoded message to FluentBridge for cross-chain delivery.
    // msg.value is passed through as the bridge fee paid by the caller.
    FluentBridge(getBridgeContract()).sendMessage{value:
        ↪ msg.value}(getOtherSideGateway(), message);
}

```

This contrasts with `NativeGateway.sendNativeTokens`, where the only intervening call is a view on the known bridge contract, there, via-ir successfully hoists `getBridgeContract()` to a single SLOAD.

Impact

Every ERC20 bridge transfer pays this overhead. This is the primary path for token deposits and withdrawals and affects every user bridging ERC20 assets in either direction.

Tool Used

Manual Review

Recommendation

Cache both addresses at the top of the function so the values are carried on the stack across the external-call barrier.

Issue L-9: Missing unchecked block on decrement loops in Rollup [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/67>

Vulnerability Detail

Two decrement loops in `Rollup.sol` – the `forceRevertBatch` iteration and the associated batch cleanup – use a checked `i--` even though the loop condition `i > toBatchIndex` guarantees the decrement can never underflow:

```
// Safety check: finalized batches are immutable and must never be rolled
↳ back
for (uint256 i = lastAcceptedBatchIndex; i > toBatchIndex; i--) {
    require($_batches[i].status != BatchStatus.Finalized,
        ↳ BatchAlreadyFinalized(i));
}
[.]
for (uint256 i = lastAcceptedBatchIndex; i > toBatchIndex; i--) {
    totalIncentiveFees +=
        ↳ _processForceRevertChallenged($_batchChallengedBlocks[i], fee);
    _cleanupForceRevertBatch(i);
}
```

Via-ir eliminates the overflow check on forward `for (uint256 i = 0; i < array.length; ++i)` loops because the bound `i < 2256` is statically provable, but it cannot prove `i > 0` at decrement time for these loops, so the condition `i > toBatchIndex` is only known at runtime. The optimized IR for these two sites still contains the checked `decrement_uint256` helper.

Impact

Affects `forceRevertBatch`, called by `EMERGENCY_ROLE` during rollup recovery. This is a low-frequency admin path, but when it fires it iterates over every batch from `lastAcceptedBatchIndex` down to `toBatchIndex`, so the per-iteration savings matter if a recovery operation unwinds many batches at once.

Tool Used

Manual Review

Recommendation

Move the decrement into an unchecked block:

```

for (uint256 i = lastAcceptedBatchIndex; i > toBatchIndex;) {
    unchecked { --i; }
}

# Issue L-10: Redundant getOtherSideGateway check in _sendOriginTokens [RESOLVED]

Source: https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/68

## Vulnerability Detail
The `sendTokens` checks `getOtherSideGateway() != address(0)` at L101, then
→ `_sendOriginTokens` re-checks the same condition at L135-136:
```solidity
function sendTokens(address token, address to, uint256 amount) external payable
 → nonReentrant {
 // Ensure the remote gateway has been configured - cannot route messages
 → without a destination
 require(getOtherSideGateway() != address(0),
 → ZeroAddressNotAllowed("getOtherSideGateway"));
[.]
function _sendOriginTokens(address token, address sender, address from, address
 → to, uint256 amount) internal returns (bytes memory) {
 // Verify remote routing is fully configured - both gateway and factory are
 → needed to
 // deterministically compute the pegged token address on the destination
 → chain
 require(
 getOtherSideGateway() != address(0) && getOtherSideFactory() !=
 → address(0),
 ZeroAddressNotAllowed("getOtherSideGateway or getOtherSideFactory")
);

```

## Impact

Affects every `sendTokens` call taking the origin-token path (ERC20 deposits on L1).

## Tool Used

Manual Review

## Recommendation

Pass the cached `otherSide` into `_sendOriginTokens` and reduce the internal `require` to the factory check only. Alternatively, drop the check from `sendTokens` and rely on the internal function's validation, but in that case `_sendPeggedTokens` needs the same guard added for symmetry.

# Issue L-11: The Heap.\_swap re-reads storage after write [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-fluent-mar-20th-2026/issues/69>

## Vulnerability Detail

The heap swap helper writes new values into `self.data[a]` and `self.data[b]`, then reads them back from storage to update the index mapping:

```
function _swap(HeapStorage storage self, mapping(bytes32 => uint256) storage
 ↪ commitmentQueueIndex, uint256 a, uint256 b) private {
 // Classic three-variable swap of the commitment hashes
 bytes32 tmp = self.data[a];
 self.data[a] = self.data[b];
 self.data[b] = tmp;

 // Keep the index mapping in sync so remove() can locate elements in O(1)
 commitmentQueueIndex[self.data[a]] = a + 1;
 commitmentQueueIndex[self.data[b]] = b + 1;
 }
```

After the swap, the new values at `self.data[a]` and `self.data[b]` are already known on the stack, as they are exactly what was just written.

## Impact

Affects every challenge queue operation - `pushChallenge`, `popChallenge`, and `_removeChallengeFromQueue`. Frequency is low in normal operation but scales with challenge activity during active disputes or slashing scenarios.

## Tool Used

Manual Review

## Recommendation

Cache both values before swapping and reuse them for the index update.



# Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.